Facultad de Ingeniería

CA3 Scope–Py

Manual de Usuario y Guía de Laboratorio

Código E1501 — Segundo Cuatrimestre

Osciloscopio Virtual Multicanal en Python para Control de Procesos



Cátedra: Control Automático III (E1501)

Departamento: Departamento de Electrotecnia

Institución: Facultad de Ingeniería — Universidad Nacional de La Plata

Año Lectivo: 2026

Versión del Manual: v1.0.0 (2026) | **Fecha de revisión:** Septiembre de 2026

Distribución oficial para alumnos disponible en el Moodle de la cátedra.

Índice general

1. Introducción y Arquitectura **1**
- 1.1. ¿Qué es CA3Scope-Py? . . . 1
- 1.2. ¿Qué es y qué no es? . . . 2
- 1.3. Propósito Didáctico en la Cursada . . . 2
- 1.4. Arquitectura del Sistema . . . 3

2. Instalación y Distribución Oficial **5**
- 2.1. Requisitos del Sistema . . . 5
- 2.2. Instrucciones de Instalación y Ejecución para el Alumno . . . 5
- 2.3. Advertencia de Windows SmartScreen o Antivirus . . . 6

3. Inicio Rápido en 5 Minutos (Modo Simulador) **7**
- 3.1. Guía Paso a Paso . . . 7

4. Descripción Detallada de la Interfaz Gráfica **9**
- 4.1. Estructura Espacial de la GUI . . . 9
- 4.2. Menú Superior y Gestión de Temas Visuales . . . 10
- 4.3. Panel 1: Conexión y Configuración . . . 11
- 4.4. Panel 2: Control de Adquisición . . . 12
- 4.5. Panel 3: Visualización . . . 12
- 4.6. Panel 4: Información y Log de Eventos . . . 13

5. Conexión del Hardware Real y Protocolos de Comunicación **14**
- 5.1. Comunicación Serie USB-UART . . . 14
- 5.2. Protocolos de Transmisión Soportados . . . 14
  - 5.2.1. Protocolo ASCII (CSV Plano) . . . 14
  - 5.2.2. Protocolo Binario 16-bit con Cabecera de Sincronización . . . 15
- 5.3. Firmwares de Referencia para Microcontroladores AVR . . . 16
  - 5.3.1. 1. Firmware ASCII (Muestreo a 50 Hz, $T_s = 20\,\text{ms}$) . . . 16
  - 5.3.2. 2. Firmware Binario Avanzado con Control PID (Muestreo a 1000 Hz, $T_s = 1\,\text{ms}$) . . . 16
- 5.4. Compilación y Grabado con PlatformIO . . . 16
- 5.5. Relación entre Fs de la GUI y Ts del Firmware . . . 17
- 5.6. Resolución de Problemas Frecuentes de Conexión . . . 17

6. Adquisición y Gestión de Datos **18**
- 6.1. Ciclo de Vida y Estados de Adquisición . . . 18
- 6.2. Capacidad y Comportamiento del Buffer . . . 19
- 6.3. Marcadores de Reanudación tras Pausa . . . 19
- 6.4. Monitoreo de Calidad y Decodificación del Enlace Serie . . . 20

7. Modos de Visualización y Ajuste Gráfico **21**
7.1. Modos de Pantalla: Superpuesto vs Subplots Separados . . . 21
7.2. Ventana Temporal y AutoScroll Horizontal . . . 22
7.3. Escala Vertical (Autoscale vs Límites Manuales) . . . 22
7.4. Personalización de Nombres de Canales . . . 23
7.5. Tasa de Refresco y Algoritmo de Decimación Gráfica . . . 23

8. Cursores Interactivos y Análisis Estadístico **24**
8.1. Sistema de Cursores Interactivos . . . 24
8.2. Origen de Datos para Estadísticas . . . 26
8.3. Tabla de Métricas Automáticas . . . 26
8.4. Estimación de Frecuencia por Cruce por Cero con Histéresis . . . 26
8.5. Significado Físico de la Desviación Estándar (σ) como Rizado AC . . . 27

9. Modo Trigger y Sincronización Avanzada **28**
9.1. El Motor de Trigger (`TriggerEngine`) . . . 28
9.2. Parámetros de Configuración del Trigger . . . 29
9.3. Mecanismo de Anclaje de Ventana y Pre-trigger . . . 29
9.4. Modos de Disparo: NORMAL vs SINGLE . . . 30
9.5. Indicadores de Estado del Trigger (Badges) . . . 30
9.6. Botones de Acción de Trigger . . . 30

10. Exportación de Datos y Capturas **31**
10.1. El Diálogo de Exportación . . . 31
10.2. Formatos de Archivo Soportados . . . 31
10.3. Opciones de Recorte de Rango Temporal . . . 32
10.4. Script de Importación y Análisis en MATLAB . . . 32

11. Técnicas de Mediciones y Metodología de Trabajo **34**
11.1. Fichas de Técnica Instrumental . . . 34
11.2. Mapeo: Necesidad de Medición vs Herramienta del Instrumento . . . 35
11.3. Recomendaciones según Temática de la Asignatura . . . 36

12. Preguntas Frecuentes y Resolución de Problemas **37**
12.1. Problemas Frecuentes y Soluciones (Troubleshooting) . . . 37
12.2. Resumen de Preguntas Frecuentes (FAQ) . . . 37

A. Guía para Desarrolladores (Instalación desde Código Fuente) **39**
A.1. Requisitos de Desarrollo . . . 39
A.2. Configuración del Entorno de Desarrollo . . . 39
A.3. Construcción del Ejecutable Portable con PyInstaller . . . 40
A.4. Notas de implementación del Buffer y Gráficas . . . 40

Índice de figuras

1.1. Vista general de la interfaz de CA3Scope-Py visualizando canales superpuestos en tiempo real. . . . 2
1.2. Diagrama de flujo del sistema físico y de datos: desde la planta hasta el análisis posterior. . . . 3
1.3. Arquitectura general multi-hilo de CA3Scope-Py: de la adquisición física al renderizado de interfaz. . . . 3

3.1. Configuración inicial en el Panel 1 seleccionando el puerto SIMULADOR a 50 Hz. 7
3.2. Panel 2 mostrando la adquisición en ejecución activa. . . . 8

4.1. Estructura espacial e identificación de columnas principales de la interfaz. . . . 10
4.2. Comparativa de visualización de la interfaz en Modo Claro y Modo Oscuro. . . . 11

5.1. Estructura comparativa de tramas de comunicación en CA3Scope-Py: a) Formato ASCII (CSV) y b) Formato Binario 16-bit con cabecera de resincronización `0xAA 0x55`. . . . 14

6.1. Detalle de los botones de control de estado en el Panel 2. . . . 18
6.2. Línea vertical punteada de color gris indicando el instante de reanudación del plotter. . . . 19

7.1. Panel 3 de Configuración de Visualización. . . . 21
7.2. Diálogo modal de asignación flexible de canales a subplots. . . . 22

8.1. Panel de Mediciones y Cursores mostrando las métricas automáticas y lectura de cursores. . . . 25

9.1. Controles del Panel de Trigger integrados en el Panel 2. . . . 28
9.2. Mecanismo de anclaje de ventana temporal y cruce por nivel del Motor de Trigger. 29
9.3. Pantalla del osciloscopio en estado Disparado (TRIG'D) mostrando los límites de pre-trigger y la línea horizontal de nivel. . . . 30

10.1. Diálogo modal de exportación seleccionando formato de salida y rango temporal. 31
10.2. Gráfica generada en MATLAB al importar y plotear el archivo .mat con el script de carga. . . . 33

Índice de tablas

1.1. Mapeo conceptual del alcance instrumental de CA3Scope-Py. 2

4.1. Parámetros y controles del Panel 1 (Conexión y Configuración). 12
4.2. Controles del Panel 2 (Control de Adquisición). 12
4.3. Controles del Panel 3 (Visualización). 13

5.1. Comparativa técnica entre Protocolo ASCII y Protocolo Binario 16-bit. 15

8.1. Métricas calculadas automáticamente por el motor estadístico (`MetricsEngine`). 26

9.1. Parámetros del Motor de Trigger. 29

11.1. Matriz de relación entre necesidades de medición y herramientas de CA3Scope-Py. 36

Capítulo 1

Introducción y Arquitectura

1.1. ¿Qué es CA3Scope-Py?

CA3Scope-Py es un osciloscopio y registrador virtual multicanal de alto rendimiento diseñado para recibir, a través de un puerto serie USB, las muestras analógicas enviadas en tiempo real por un microcontrolador. La aplicación convierte estas muestras de texto en curvas gráficas dinámicas, calcula métricas estadísticas de forma automática y permite exportar los datos a archivos compatibles con herramientas de análisis en cualquier computadora.

El desarrollo del software hace foco en simplificar la visualización y adquisición de señales desde una perspectiva amigable para el estudiante, superando las limitaciones de los terminales serie de texto clásicos. De esta manera, se facilita a los alumnos probar y diseñar sus algoritmos tanto en el laboratorio como desde sus hogares mediante el uso de placas de desarrollo de bajo costo o utilizando el simulador de señales integrado.

Es importante destacar que esta herramienta no reemplaza la experiencia de trabajo en el laboratorio de control —el contacto directo con las plantas físicas e instrumentos de banco sigue siendo el núcleo pedagógico de la cursada—, ni pretende competir en ancho de banda o precisión con un osciloscopio de oscilaciones rápidas. Debido a las limitaciones de velocidad del enlace serie y los convertidores ADC de los microcontroladores de desarrollo, CA3Scope-Py está orientado específicamente a ensayos de control automático, tales como la medición de transitorios lentos, oscilaciones en régimen, saturación de actuadores y rizado de señales.

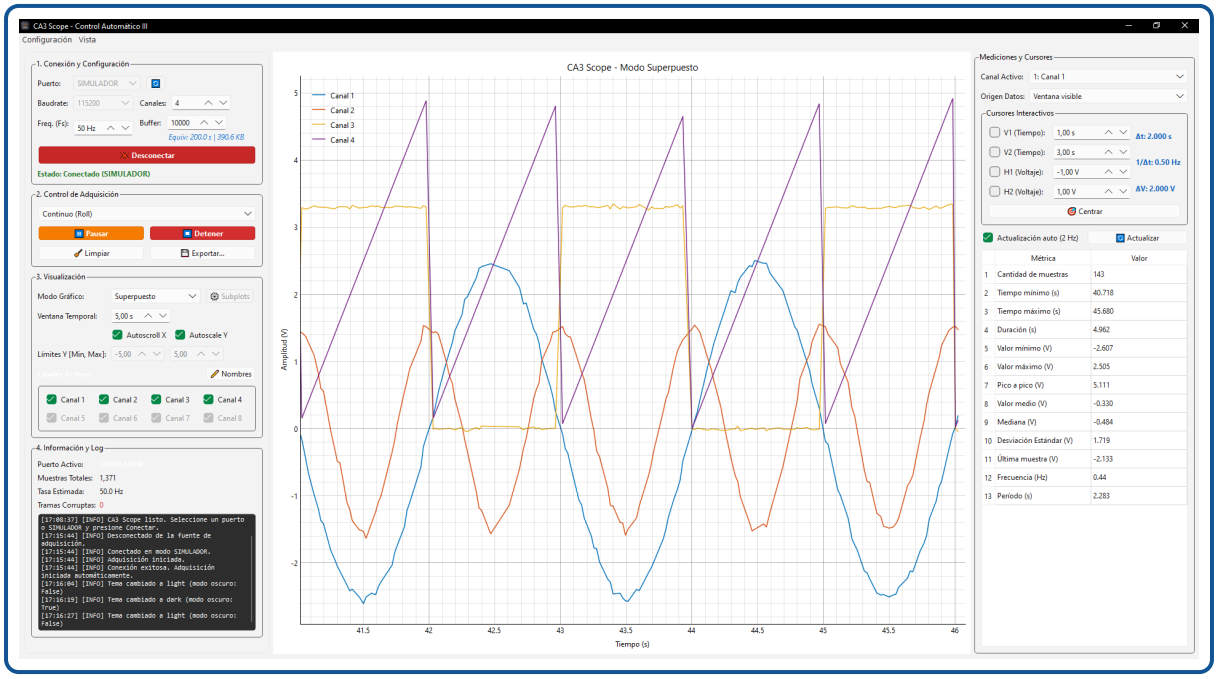


Figura 1.1: Vista general de la interfaz de CA3Scope-Py visualizando canales superpuestos en tiempo real.

1.2. ¿Qué es y qué no es?

Para evitar confusiones sobre el alcance del instrumento durante el desarrollo de los trabajos prácticos de la cursada, la Tabla 1.1 resume los objetivos de diseño y las limitaciones del sistema.

Tabla 1.1: Mapeo conceptual del alcance instrumental de CA3Scope-Py.

CA3Scope-Py SÍ es...	**CA3Scope-Py NO es...**
Un registrador y graficador virtual en tiempo real para señales transitorias lentas.	Un reemplazo para las plantas físicas del laboratorio ni para el instrumental de banco.
Una herramienta de visualización y exportación de variables de control (referencia, salida, control).	Un osciloscopio de alta frecuencia (su ancho de banda está limitado a menos de 1 kHz).
Un software complementario para sintonía y validación práctica de algoritmos PID, MPC o lógica de relé.	Un analizador de espectros o instrumento de medición de precisión metrológica industrial.
Un entorno de simulación interna para estudiar el instrumental y las métricas sin hardware conectado.	Un entorno cerrado de programación de microcontroladores (no compila ni graba código).

1.3. Propósito Didáctico en la Cursada

El objetivo fundamental de la herramienta es brindar a los alumnos de 4to año una plataforma versátil y transparente que les permita:

- **Caracterizar plantas y sistemas reales:** Medir respuestas al escalón, frecuencias de oscilación en ciclos límite y respuestas en frecuencia.
- **Sintonizar y validar controladores en tiempo real:** Observar la acción de control ($u(t)$) y la variable de proceso ($y(t)$) producidas por microcontroladores como Arduino, ESP32 o STM32.
- **Operar de forma remota o en casa:** Gracias a su simulador de señales integrado, los alumnos pueden realizar las actividades prácticas desde sus computadoras personales sin necesidad de disponer de instrumental físico.

1.4. Arquitectura del Sistema

El ecosistema de medición completo abarca tanto elementos de hardware físico como módulos de software. La Figura 1.2 ilustra la cadena de adquisición y el flujo de los datos desde la planta hasta el análisis posterior en herramientas de ingeniería.

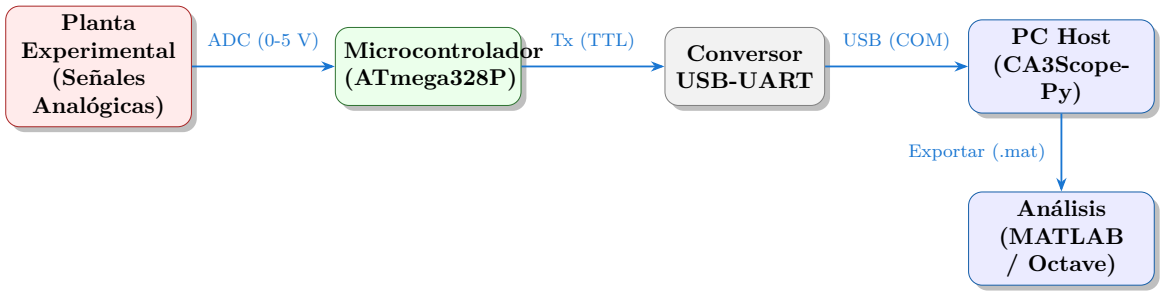


Figura 1.2: Diagrama de flujo del sistema físico y de datos: desde la planta hasta el análisis posterior.

Por su parte, la arquitectura interna del software en la PC está diseñada de forma multi-hilo para garantizar que la interfaz gráfica permanezca fluida y sin bloqueos durante la recepción masiva de datos serie. Como se detalla en la Figura 1.3, el backend y el frontend se comunican a través de señales seguras de Qt.

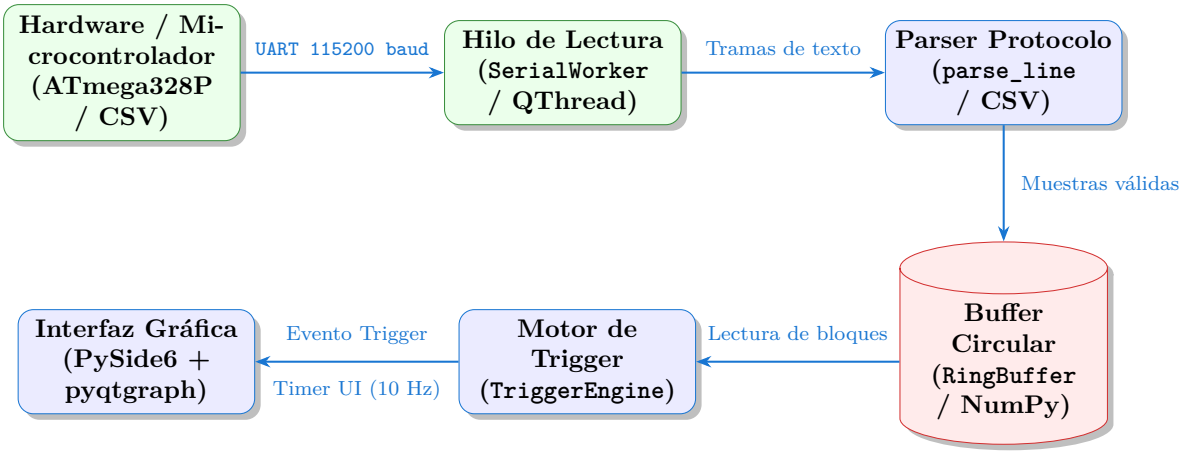


Figura 1.3: Arquitectura general multi-hilo de CA3Scope-Py: de la adquisición física al renderizado de interfaz.

1. **Hardware / Emisor (Planta o Microcontrolador):** Adquiere las señales analógicas a través de convertidores ADC de 10 o 12 bits y las transmite como tramas formateadas en ASCII (CSV) o Binario 16-bit empacado por un enlace USB-UART (puerto serie).

2. **Motor Core y Buffers (PC Backend):** Un hilo secundario dedicado (`SerialWorker`) lee de forma asíncrona y no bloqueante los bytes del puerto serie, calcula marcas temporales monotónicas de alta precisión (`time.monotonic()`), procesa y valida las tramas según el protocolo configurado (`parse_line` para ASCII o `parse_binary_frame` con resincronización de cabecera `0xAA 0x55` para Binario 16-bit), y deposita los datos en un buffer circular matricial NumPy (`RingBuffer`).

3. **Interfaz Gráfica y Plotter (GUI Frontend):** Un temporizador principal Qt refresca la pantalla a 10 Hz extrayendo bloques del buffer circular, actualiza los marcadores de pausa y renderiza los cursores y la tabla de estadísticas automáticas.

Nota Toda la lógica de adquisición, filtrado, disparo y cálculo de métricas en CA3Scope-Py ha sido implementada vectorialmente con NumPy, garantizando un consumo mínimo de CPU incluso con tasas de muestreo de hasta 1000 Hz por canal.

Capítulo 2

Instalación y Distribución Oficial

2.1. Requisitos del Sistema

CA3Scope-Py en su versión pre-compilada para estudiantes no requiere la instalación previa de Python, intérpretes adicionales ni permisos de administrador en el sistema operativo.

- **Sistema Operativo Compatibilidad Principal:** Microsoft Windows 10 u 11 (64-bit).
- **Almacenamiento:** 150 MB libres en disco rígido o unidad ejecutable portable (pendrive USB).
- **Permisos:** No requiere permisos de administrador para su ejecución.

2.2. Instrucciones de Instalación y Ejecución para el Alumno

El canal principal y oficial de distribución para los alumnos de la asignatura es la plataforma Moodle de la cátedra.

1. **Descarga:** Descargá el archivo comprimido oficial (`CA3Scope.zip`) desde el aula virtual Moodle de Control Automático III.
2. **Descompresión Completa (Paso Obligatorio):** Descomprimí el contenido de la carpeta comprimida en la ubicación deseada de tu computadora (por ejemplo, en el Escritorio, Documentos o en un Pendrive USB).
3. **Ejecución:** Ingresá a la carpeta descomprimida y ejecutá directamente el archivo principal:

`CA3Scope.exe`

Advertencia Importante **Pautas Críticas de Funcionamiento:**

- **No ejecutar desde el archivo zip sin descomprimir:** Ejecutar la aplicación directamente desde el explorador de archivos comprimidos impedirá la carga de las bibliotecas y provocará un cierre inesperado.
- **No separar ni mover el archivo ejecutable:** El archivo `CA3Scope.exe` requiere para funcionar la presencia de la carpeta `_internal/` ubicada en el mismo directorio. Si movés o copiás el ejecutable a otra carpeta individual, la aplicación no podrá iniciar.

2.3. Advertencia de Windows SmartScreen o Antivirus

Al tratarse de un ejecutable generado para la cursada y no disponer de una firma digital paga, es posible que el filtro Windows SmartScreen o tu antivirus muestre un cartel de advertencia la primera vez que intentes abrir el programa (*"Windows protegió su PC"*).

Nota **Cómo continuar de forma segura:**

1. En la ventana flotante de Windows SmartScreen, hacé clic en el enlace **"Más información"**.
2. Se habilitará un nuevo botón en la parte inferior. Hacé clic en **"Ejecutar de todos modos"**.

La aplicación se abrirá normalmente y Windows recordará tu elección para futuras ejecuciones.

Consejo Práctico Si deseás consultar los detalles de instalación desde el código fuente o compilar tu propio ejecutable, consultá el **Anexo A: Guía para Desarrolladores**.

Capítulo 3

Inicio Rápido en 5 Minutos (Modo Simulador)

Este capítulo guía al estudiante en su primer contacto con el programa utilizando el **Simulador Interno de Señales**, sin necesidad de conectar placas ni hardware externo.

3.1. Guía Paso a Paso

1. **Iniciar la aplicación:** Abrí el ejecutable `CA3Scope.exe` desde la carpeta descomprimida. Se desplegará instantáneamente una pantalla de bienvenida (*Splash Screen*) con el logo oficial y una barra de progreso animada mientras se inicializan los módulos principales.
2. **Seleccionar el puerto SIMULADOR:** En el **Panel 1 (Conexión y Configuración)** ubicado a la izquierda de la pantalla, verificá que la casilla *Puerto* tenga seleccionado el valor `SIMULADOR`.

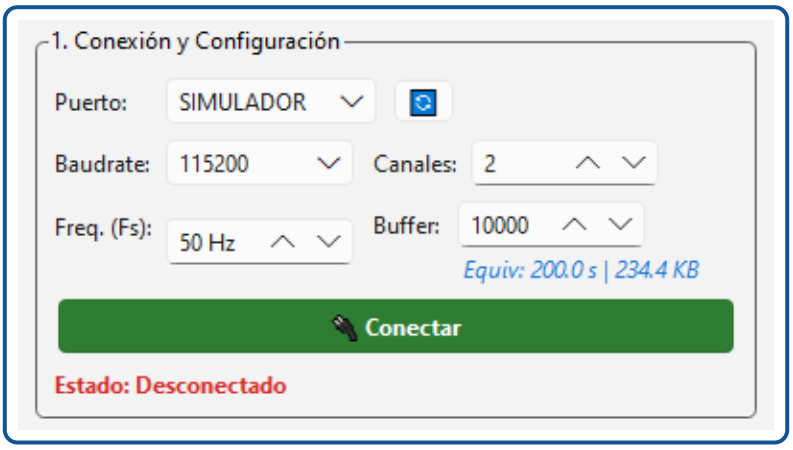


Figura 3.1: Configuración inicial en el Panel 1 seleccionando el puerto SIMULADOR a 50 Hz.

3. **Establecer parámetros de adquisición:**
   - **Baudrate:** `115200` (ignorado internamente en simulación).
   - **Canales:** `2` (por defecto genera Seno 0.5 Hz en Ch1 y Coseno 1.0 Hz en Ch2).
   - **Freq. (Fs):** `50 Hz`.
   - **Buffer:** `10000` muestras (equivalente a 200 segundos de capacidad).

4. **Conectar:** Hacé clic en el botón verde `[Conectar]`. El indicador de estado cambiará a color verde mostrando **Estado: Conectado (SIMULADOR)**.

5. **Iniciar Adquisición:** En el **Panel 2 (Control de Adquisición)**, hacé clic en el botón azul `[Iniciar]`. Verás inmediatamente las señales sintéticas desplazándose suavemente de derecha a izquierda por el osciloscopio.



Figura 3.2: Panel 2 mostrando la adquisición en ejecución activa.

6. **Probar la Pausa:** Presioná el botón naranja `[Pausar]`. La gráfica se congelará instantáneamente. Al presionar `[Reanudar]`, la adquisición continuará y se trazará una línea vertical punteada de color gris marcando la posición de reanudación, manteniendo la coherencia temporal al descontar la duración de la pausa.

7. **Activar los Cursores de Medición:** En la columna derecha, dentro del panel **Mediciones y Cursores**, activá las casillas de verificación `V1 (Tiempo)` y `V2 (Tiempo)`. Aparecerán dos cursores verticales amarillos sobre la gráfica. Arrastralos con el mouse para medir el período de la onda senoidal; el panel actualizará automáticamente los valores de Δt y $1/\Delta t$.

Consejo Práctico Si deseás reiniciar la vista de la gráfica en cualquier momento sin perder la conexión, presioná el botón `[Limpiar]` en el Panel de Adquisición.

Capítulo 4

Descripción Detallada de la Interfaz Gráfica

La ventana principal de CA3Scope-Py ha sido estructurada en tres columnas principales divididas por paneles deslizantes (*QSplitter*), optimizando el espacio en pantalla para monitoreo y análisis simultáneo.

4.1. Estructura Espacial de la GUI

La distribución de los componentes en la pantalla se compone de:

1. **Barra de Menús Superior:** Acceso a la carga/guardado de configuraciones JSON y selección de temas visuales.
2. **Columna Izquierda (Controles de Adquisición y Visualización):** Agrupa en una sección desplazable verticalmente (*QScrollArea*) los Paneles 1, 2, 3 y 4.
3. **Columna Central (Osciloscopio Principal):** Lienzo gráfico interactivo desarrollado sobre `pyqtgraph.PlotWidget`.
4. **Columna Derecha (Mediciones y Cursores):** Gestión de cursores V_1, V_2, H_1, H_2 y estadísticas automáticas.

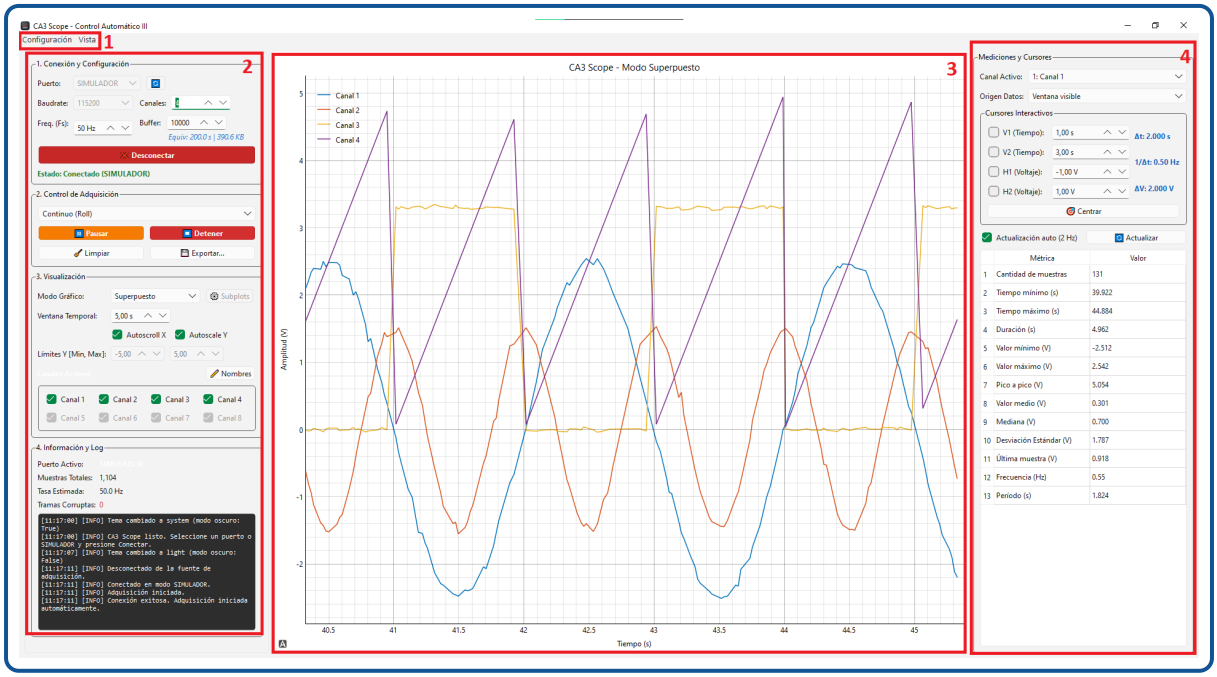


Figura 4.1: Estructura espacial e identificación de columnas principales de la interfaz.

4.2. Menú Superior y Gestión de Temas Visuales

En la barra de menú superior se encuentran las siguientes opciones:

- **Menú Configuración:**
  - `Cargar Configuración...`: Importa un archivo JSON con presets de canales, colores, escala Y y modos de trigger.
  - `Guardar Configuración...`: Exporta el estado actual de la GUI a un archivo JSON.
- **Menú Vista:**
  - `Mostrar panel de control`: Muestra u oculta la columna izquierda.
  - `Mostrar panel de mediciones`: Muestra u oculta la columna derecha.
  - `Tema de la aplicación`: Permite alternar dinámicamente entre tres esquemas de color:
    1. **Predeterminado del sistema:** Adapta los colores según la apariencia global del sistema operativo.
    2. **Modo Claro:** Fondo blanco puro, cuadrícula tenue y curvas de alto contraste.
    3. **Modo Oscuro:** Fondo gris oscuro (`#1e1e1e`), ideal para entornos de baja iluminación en laboratorios.
- **Menú Ayuda:**
  - `Acerca de CA3 Scope...`: Abre el diálogo modal institucional estructurado en cuatro pestañas (**Software**, **Universidad y Cátedra**, **Plantel Docente**, **Licencia y Créditos**) con adaptación dinámica de color (claro/oscuro). Incluye el botón **`[Abrir Manual de Usuario (PDF)]`** que abre la versión oficial del manual en el navegador web.



Figura 4.2: Comparativa de visualización de la interfaz en Modo Claro y Modo Oscuro.

4.3. Panel 1: Conexión y Configuración

El Panel 1 configura los parámetros fundamentales del enlace de comunicación serie, selección de protocolo (ASCII o Binario 16-bit) y la reserva de memoria para el buffer circular.

Tabla 4.1: Parámetros y controles del Panel 1 (Conexión y Configuración).

Control GUI	Rango / Opciones	Default	Efecto en el Sistema
Puerto (QComboBox)	SIMULADOR, COM	SIMULADOR	Selecciona la fuente de datos serie o simulación interna.
Refrescar (QPushButton)	[Refrescar]	—	Reescanea los puertos COM USB-UART disponibles.
Formato (QComboBox)	ASCII, Binario 16-bit	ASCII	Define la decodificación del flujo serie: texto CSV o binario de 16 bits empacado.
Baudrate (QComboBox)	9600 a 460800	115200	Ajusta la velocidad en baudios de la UART serie.
Canales (QSpinBox)	1 a 8	2	Define la cantidad de variables a adquirir por trama.
Freq. Fs (QSpinBox)	1 a 1000 Hz	50 Hz	Frecuencia de muestreo esperada para cálculo de base de tiempo.
Buffer (QSpinBox)	100 a 1e6	10000	Máxima cantidad de muestras retenidas en el RingBuffer.
Equivalencia (QLabel)	Lectura informativa	Dinámico	Muestra la capacidad en segundos y Megabytes de RAM reservada.
Conectar (QPushButton)	[Conectar] / [Desconectar]	Desconectado	Abre/cierra el puerto o inicia el simulador.

4.4. Panel 2: Control de Adquisición

El Panel 2 gestiona los comandos de flujo continuo y modo de funcionamiento.

Tabla 4.2: Controles del Panel 2 (Control de Adquisición).

Control GUI	Rango / Opciones	Default	Efecto en el Sistema
Modo Captura (QComboBox)	Continuo (Roll), Disparado (Trigger)	Continuo	Cambia entre flujo continuo o motor de disparo por nivel.
Run/Pausa (QPushButton)	[Iniciar] / [Pausar] / [Reanudar]	Iniciar	Controla el ciclo de vida de la lectura de muestras.
Detener (QPushButton)	[Detener]	Deshabilitado	Detiene por completo los hilos de adquisición.
Limpiar (QPushButton)	[Limpiar]	Habilitado	Vacía el buffer circular y resetea contadores a cero.
Exportar (QPushButton)	[Exportar...]	Habilitado	Abre el diálogo modal de exportación a CSV/MAT/PNG.

4.5. Panel 3: Visualización

Permite adaptar los aspectos estéticos y de escala de las curvas en el osciloscopio.

Tabla 4.3: Controles del Panel 3 (Visualización).

Control GUI	Rango / Opciones	Default	Efecto en el Sistema
Modo Gráfico (`QComboBox`)	Superpuesto, Subplots separados	Superpuesto	Grafica todos los canales en 1 eje o en ejes separados.
Subplots (`QPushButton`)	`[Subplots]`	Deshabilitado	Abre el diálogo de asignación de canales por subplot.
Ventana Temporal (`QDoubleSpinBox`)	0.5 s a 300.0 s	`5.0 s`	Ancho visible del eje X en segundos.
Autoscroll X (`QCheckBox`)	ON / OFF	Activado	Desplaza automáticamente la vista hacia la última muestra.
Autoscale Y (`QCheckBox`)	ON / OFF	Activado	Ajusta dinámicamente los límites verticales según los datos.
Límites Y (`QDoubleSpinBox`)	±1e9	[-5.0, 5.0]	Fija manualmente los límites verticales si Autoscale Y está OFF.
Canales Activos (`QCheckBox`)	Ch 1 a Ch 8	Todos ON	Muestra u oculta individualmente cada curva gráfica.
Renombrar (`QPushButton`)	`[Nombres]`	Habilitado	Personaliza las etiquetas del sistema (ej: "PWM", "Vout").

4.6. Panel 4: Información y Log de Eventos

Ubicado en la parte inferior de la columna izquierda, este panel muestra diagnósticos en tiempo real:

- **Puerto Activo:** Muestra el puerto en uso (ej. `COM3` o `SIMULADOR`).
- **Muestras Totales:** Contador formateado del total de muestras ingresadas al buffer.
- **Tasa Estimada:** Frecuencia real de recepción calculada en la PC en Hz.
- **Tramas Corruptas:** Contador en texto rojo de tramas descartadas por errores de sintaxis o formato.
- **Área de Log:** Consola de texto que registra advertencias de conexión, aperturas de puertos y errores.

Capítulo 5

Conexión del Hardware Real y Protocolos de Comunicación

5.1. Comunicación Serie USB-UART

CA3Scope-Py se comunica con placas de desarrollo basadas en microcontroladores AVR (como ATmega328P [4] en Arduino UNO/Nano), ESP32, STM32 o adaptadores USB-UART (FTDI, CH340, CP2102) a través de un puerto serie virtual.

La comunicación es asíncrona y soporta dos protocolos de transmisión configurables desde la interfaz gráfica: **Formato ASCII (CSV)** y **Formato Binario 16-bit**. Ambos delimitados por un salto de línea (`\n` o `\r\n`).

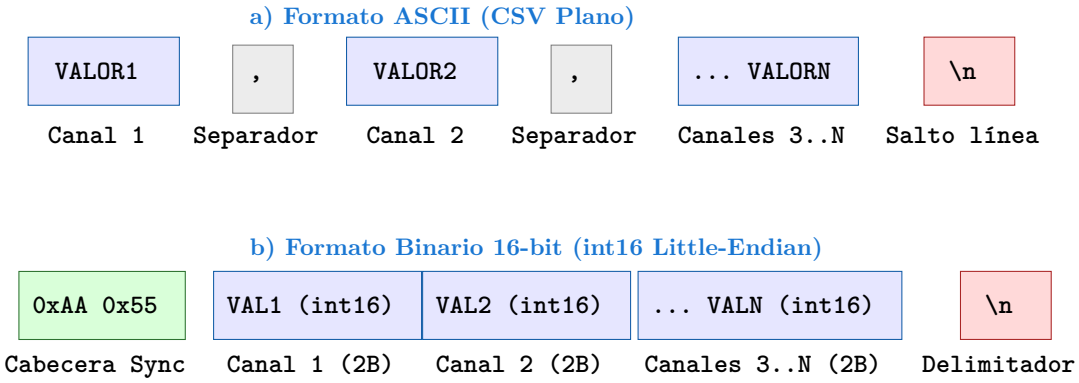


Figura 5.1: Estructura comparativa de tramas de comunicación en CA3Scope-Py: a) Formato ASCII (CSV) y b) Formato Binario 16-bit con cabecera de resincronización `0xAA 0x55`.

5.2. Protocolos de Transmisión Soportados

5.2.1. Protocolo ASCII (CSV Plano)

El formato ASCII transmite los valores de las muestras como texto legible separado por comas:

```
VALOR1,VALOR2,VALOR3,...,VALORN\n
```

- **Estructura:** Cada valor es una cadena que representa un número (entero o flotante, ej: `2.500,1.230\n`).

- **Ventajas:** Extrema simplicidad, transparencia e inmediatez para depurar manualmente con cualquier monitor serie (Arduino IDE, Serial Monitor, PuTTY).
- **Limitaciones:** Elevada sobrecarga en el ancho de banda del puerto serie (cada muestra ocupa entre 20 y 50 bytes) y alto consumo de tiempo de CPU en el microcontrolador al ejecutar funciones de conversión de texto como `snprintf()` o `dtostrf()`. Apto para frecuencias de muestreo bajas o moderadas ($F_s \leq 100\,\text{Hz}$).

5.2.2. Protocolo Binario 16-bit con Cabecera de Sincronización

Para aplicaciones de control de alta velocidad (frecuencias de muestreo hasta 1000 Hz, $T_s = 1\,\text{ms}$), CA3Scope-Py implementa un protocolo binario empacado de 16 bits por canal:

```
[0xAA] [0x55] [CH1_L] [CH1_H] [CH2_L] [CH2_H] ... [CHN_L] [CHN_H] [\n]
```

- **Cabecera de Sincronización (`0xAA 0x55`):** Dos bytes mágicos fijos al inicio de cada trama. Si el buffer serie sufre una pérdida de bytes o un desborde, la aplicación busca automáticamente este patrón en el flujo binario para resincronizar inmediatamente la alineación de canales sin desfasar las mediciones.
- **Cuerpo de Datos (Payload):** Cada canal se codifica como un entero de 16 bits firmado o sin signo (`int16_t` / `uint16_t`) enviado en formato *Little-Endian* (byte menos significativo primero). Por ejemplo, un canal ADC de 0 a 5000 mV o de 0 a 1023 unidades.
- **Delimitador Final:** Un byte de salto de línea (`\n`, `0x0A`) que marca el fin del paquete binario.
- **Ventajas:**
  1. *Mínimo tamaño de trama:* Una medición de 6 canales ocupa únicamente 15 bytes (2 de cabecera + 12 de datos + 1 de delimitador), comparado con los >40 bytes del formato ASCII.
  2. *Cero formateo en el microcontrolador:* Transmisión directa por registros UART mediante desplazamientos de bits binarios (`shift`), reduciendo el tiempo de envío de milisegundos a microsegundos en la ISR.
  3. *Soporte multicanal a* 1000 *Hz:* Permite ejecutar lazos de control PID/MPC en tiempo real determinístico a 1 kHz sin saturar la UART a 115200 bps.

Tabla 5.1: Comparativa técnica entre Protocolo ASCII y Protocolo Binario 16-bit.

Característica	**Formato ASCII (CSV)**	**Formato Binario 16-bit**
Tamaño de Trama (6 ch)	35 – 50 bytes	15 bytes fijos
Procesamiento en MCU	Alto (`snprintf` / texto)	Mínimo (`shift` de bytes)
Máx. Fs recomendada	$\leq 100\,\text{Hz}$	Hasta 1000 Hz ($T_s = 1\,\text{ms}$)
Legibilidad / Debug	Legible en texto plano	Requiere decodificador o GUI
Protección de Sincro	Salto de línea (`\n`)	Cabecera `0xAA 0x55` + Salto de línea

5.3. Firmwares de Referencia para Microcontroladores AVR

En la carpeta `firmware/` del repositorio se proveen dos proyectos completos listos para compilar en **PlatformIO** [6] (VS Code) o la cadena de herramientas `avr-gcc` y `avr-libc` [1]:

5.3.1. 1. Firmware ASCII (Muestreo a 50 Hz, $T_s = 20\,\text{ms}$)

Ubicado en `firmware/firmware_comunicacion_ascii/src/main.c`. Formatea dos canales en punto flotante y los transmite como texto separado por comas mediante un Timer en modo CTC.

5.3.2. 2. Firmware Binario Avanzado con Control PID (Muestreo a 1000 Hz, $T_s = 1\,\text{ms}$)

Ubicado en `firmware/firmware_comunicacion_binaria/src/main.c`. Ejecuta un algoritmo de control PID discreto optimizado dentro de la ISR de interrupción del Timer 1 (cada 1 ms), lee 5 canales ADC y la acción de control PWM, y transmite la trama binaria empacada con cabecera `0xAA 0x55` a 115200 baudios.

A continuación se ilustra el fragmento central de transmisión binaria en C para microcontrolador:

```
// Transmision UART en formato binario 16-bit con cabecera 0xAA 0x55
UART_transmit(0xAA); // Byte 1 de Sincronizacion
UART_transmit(0x55); // Byte 2 de Sincronizacion

// Envio de 5 Canales ADC + 1 Canal PWM en int16 Little-Endian (2 bytes/valor)
UART_write_uint16(v0);
UART_write_uint16(v1);
UART_write_uint16(v2);
UART_write_uint16(v3);
UART_write_uint16(v4);
UART_write_uint16(pwm);

UART_transmit('\n'); // Byte delimitador de fin de trama
```

Donde la función helper de transmisión de 16 bits en C se implementa como:

```
void UART_write_uint16(uint16_t val) {
    UART_transmit((uint8_t)(val & 0xFF));        // Byte menos significativo (LSB)
    UART_transmit((uint8_t)((val >> 8) & 0xFF)); // Byte mas significativo (MSB)
}
```

5.4. Compilación y Grabado con PlatformIO

Para compilar y subir cualquiera de los firmwares provistos:

1. Abrí en VS Code la carpeta del firmware deseado:
   - `firmware/firmware_comunicacion_ascii`
   - `firmware/firmware_comunicacion_binaria`
2. Ejecutá los comandos estándar de PlatformIO desde la terminal integrada:

```
# Compilar el proyecto
pio run
# Subir el firmware a la placa conectada por USB
pio run -t upload
```

5.5. Relación entre Fs de la GUI y Ts del Firmware

Advertencia Importante **Importante sobre la Base de Tiempo:** Los timestamps (en segundos) **NO provienen del microcontrolador**. Se generan exclusivamente en la PC mediante el reloj monotónico del sistema operativo al recibir cada muestra.

Para evitar distorsiones horizontales en la escala de tiempo:

1. El valor de **Freq. (Fs)** configurado en el Panel 1 de la app debe coincidir exactamente con el inverso del período de muestreo fijado en el firmware:

$$F_{s,\text{app}} = \frac{10^6}{T_{s,\mu\text{s}}} \quad [\text{Hz}]$$

2. Si el firmware binario ejecuta a $T_s = 1\,\text{ms}$ ($1000\,\mu\text{s}$), ajustá el control **Freq. (Fs)** en la interfaz a 1000 Hz y seleccioná el formato **Binario 16-bit**.

5.6. Resolución de Problemas Frecuentes de Conexión

Advertencia Importante **Bloqueo del Puerto Serie por Monitores y Entornos de Desarrollo:** Los monitores serie de herramientas de desarrollo como **PlatformIO** (Serial Monitor `pio device monitor`), **Microchip Studio / MPLAB**, **Arduino IDE** o **PuTTY** obtienen acceso exclusivo al puerto COM. Si se intenta conectar CA3Scope-Py mientras un monitor serie externo está abierto, la aplicación generará una advertencia de `PermissionError`. Es estrictamente necesario cerrar la terminal o monitor serie externo antes de hacer clic en **Conectar**.

- **"Error al abrir puerto COM3: PermissionError" / Puerto Ocupado:** Ocurre cuando un monitor serie externo (PlatformIO, Microchip Studio, Arduino IDE, PuTTY) mantiene retenido el puerto serie. Cerrá el monitor activo y reintentá la conexión.
- **Graficación de valores erráticos en modo Binario:** Verificá que la cantidad de canales configurada en el Panel 1 coincida exactamente con la cantidad de enteros de 16 bits transmitidos por el firmware.
- **Desconexión física accidental:** Si se desconecta el cable USB, el hilo `SerialWorker` notifica el evento, cierra el puerto de forma segura y actualiza el estado a *Desconectado* sin bloquear la GUI.

Capítulo 6

Adquisición y Gestión de Datos

6.1. Ciclo de Vida y Estados de Adquisición

La adquisición de datos en CA3Scope-Py es gestionada por la clase principal `SerialAcquisition`, la cual opera una máquina de estados determinística:

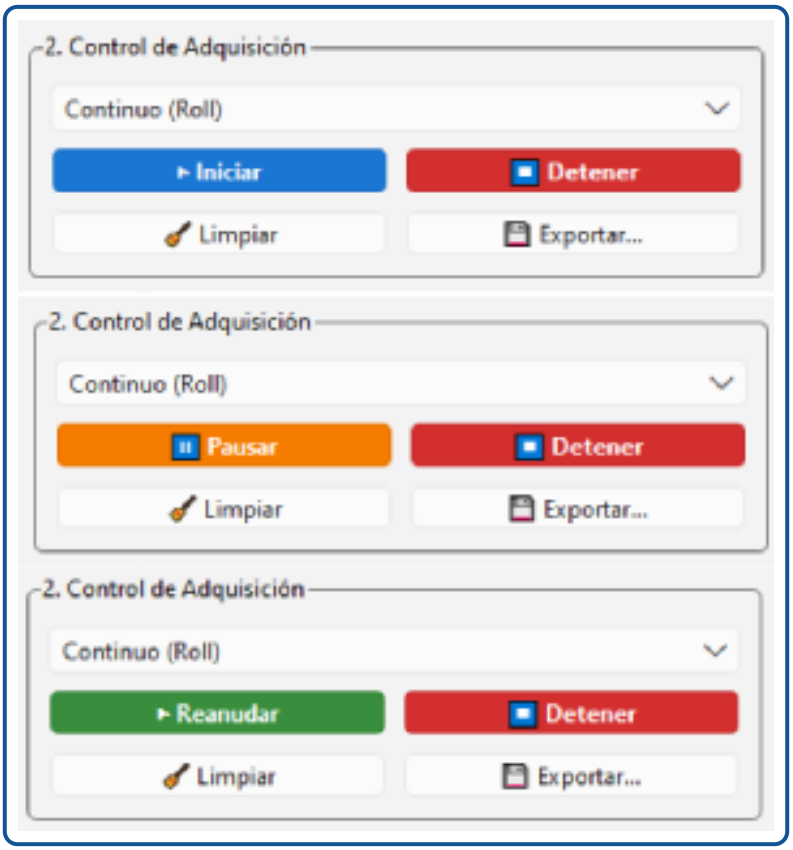


Figura 6.1: Detalle de los botones de control de estado en el Panel 2.

Los tres estados principales visibles en la adquisición son:

- **Run (Corriendo):** El hilo de recepción lee continuamente el puerto serie o genera muestras sintéticas, insertándolas en el buffer circular y emitiendo señales de refresco visual.
- **Pausa:** Detiene el flujo de emisión visual pero mantiene la fuente activa o el puerto serie

abierto. Al reanudar, la aplicación calcula la duración de la pausa mediante el administrador de marcas temporales y agrega un marcador visual de reanudación.

- **Stop (Detenido):** Cierra el puerto serie o detiene el temporizador del simulador y congela la adquisición.

6.2. Capacidad y Comportamiento del Buffer

Para permitir que la aplicación funcione de forma continua durante horas sin saturar la memoria de la PC, CA3Scope-Py almacena temporalmente los datos recibidos en un buffer circular en memoria. El usuario puede configurar el tamaño máximo de este buffer (cantidad de muestras reservadas) en la casilla **Buffer** del Panel 1.

La cantidad total de tiempo (en segundos) que el instrumento es capaz de recordar en el historial depende directamente tanto del tamaño del buffer como de la frecuencia de muestreo de las señales (F_s), calculándose de la siguiente forma:

$$T_{\text{cap}} = \frac{N_{\text{buffer}}}{F_s} \quad [\text{segundos}]$$

Por ejemplo, si se configura un tamaño de buffer de 10000 muestras y la frecuencia de muestreo se establece en 50 Hz, el sistema retendrá en memoria un historial de exactamente los últimos 200 segundos de adquisición.

- **Comportamiento al llenarse el buffer:** Cuando se alcanza el límite de capacidad de almacenamiento configurado, las nuevas muestras recibidas sobreescriben automáticamente a las muestras más antiguas. De esta manera, en pantalla y al exportar siempre se dispondrá de la ventana de tiempo más reciente de forma ininterrumpida.
- **Indicador de Equivalencia en la GUI:** En la parte inferior del Panel 1, la etiqueta informativa **Equivalencia** muestra de forma dinámica la capacidad total de almacenamiento temporal calculada en segundos, junto con el consumo estimado de memoria RAM que se reservará para esa configuración.

6.3. Marcadores de Reanudación tras Pausa

Cuando la adquisición pasa del estado **Pausa** a **Reanudar**, la interfaz gráfica dibuja una línea vertical punteada de color gris semitransparente (`RGB: 150, 150, 150`) en la posición temporal exacta donde se retomó la captura de datos. Esta marca visual ayuda a identificar las interrupciones del ensayo sin introducir discontinuidades ni saltos en la escala de tiempo de la gráfica.

Captura de Pantalla Pendiente

Archivo requerido en el repositorio:
`docs/manual/screenshots/10_marcadores_pausa.png`

Línea vertical punteada de color gris indicando el instante de reanudación del plotter.

Figura 6.2: Línea vertical punteada de color gris indicando el instante de reanudación del plotter.

6.4. Monitoreo de Calidad y Decodificación del Enlace Serie

El backend de adquisición en el hilo `SerialWorker` procesa el flujo entrante de bytes acumulándolo en un buffer bruto en RAM y aplicando el algoritmo de parseo según el formato seleccionado:

- **Decodificación ASCII (CSV):** Busca el delimitador de línea `\n`. Extrae los tokens separados por comas, los convierte a valores numéricos flotantes en NumPy y verifica su validez computacional (`math.isfinite()`).
- **Decodificación Binaria 16-bit:** Busca la cabecera de sincronización `0xAA 0x55`. Extrae el bloque exacto de $2 \times N_{\text{canales}}$ bytes de payload, desempaqueta los enteros de 16 bits little-endian con el módulo `struct` de Python y valida que el byte delimitador sea el salto de línea (`0x0A`).
- **Resincronización ante Errores:** Si ocurren pérdidas de bytes por desborde en el buffer de la UART o si la trama binaria se corrompe, el algoritmo descarta los bytes basura hasta encontrar la siguiente cabecera `0xAA 0x55` o delimitador `\n`, resincronizando el stream de datos automáticamente.

El Panel 4 (Información y Log) supervisa la salud del enlace en tiempo real mediante los siguientes contadores:

- **Muestras Totales:** Contador acumulado de muestras válidas integradas exitosamente al buffer circular.
- **Tasa Estimada (Hz):** Frecuencia real de llegada calculada mediante marcas de tiempo monotónicas en la PC.
- **Tramas Corruptas:** Contador en texto destacado rojo de tramas descartadas debido a pérdidas de caracteres, valores incompletos, desalineación binaria o texto no numérico.

Capítulo 7

Modos de Visualización y Ajuste Gráfico

7.1. Modos de Pantalla: Superpuesto vs Subplots Separados

CA3Scope-Py ofrece dos modalidades de disposición de ejes gráficos configurables desde el Panel 3:

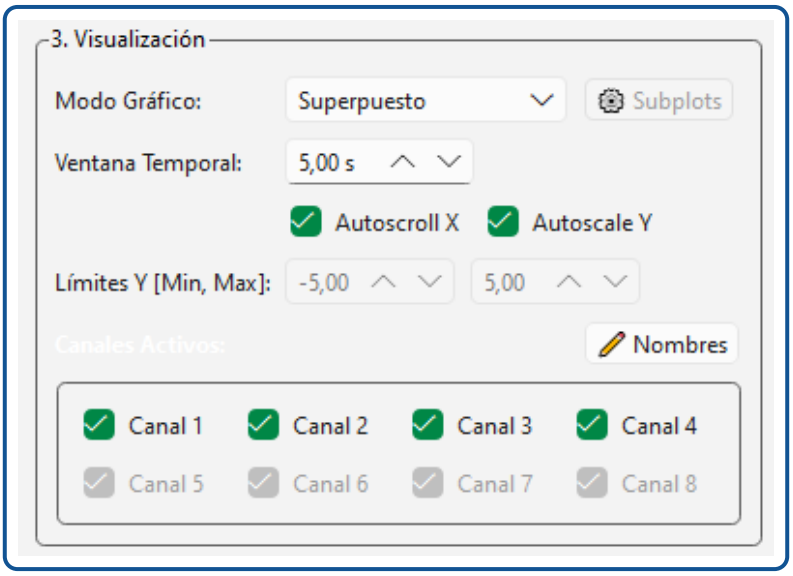


Figura 7.1: Panel 3 de Configuración de Visualización.

Modo Superpuesto (Overlay): Todos los canales activos se grafican sobre un único par de ejes cartesianos (t, V).

- **Ventajas:** Permite comparar directamente amplitudes, relaciones de fase y desfasajes temporales entre la entrada de referencia $r(t)$ y la salida $y(t)$.
- **Paleta de colores oficial:** Ch1 (Azul `#0072BD`), Ch2 (Naranja `#D95319`), Ch3 (Amarillo `#EDB120`), Ch4 (Púrpura `#7E2F8E`), Ch5 (Verde `#77AC30`), Ch6 (Celeste `#4DBEEE`), Ch7 (Rojo `#A2142F`), Ch8 (Gris `#404040`).

Modo Subplots Separados: La zona de trazado se divide en múltiples sub-gráficos alineados verticalmente que comparten el mismo eje horizontal de tiempo X.

- Habilita el botón **[Subplots]**, el cual despliega el diálogo modal de configuración de subplots.
- Permite elegir la cantidad de sub-gráficos (de 1 a $N_{canales}$) y asignar qué canales se agrupan en cada subplot (por ejemplo, Ch1 y Ch2 en Subplot 1, y Ch3 en Subplot 2).

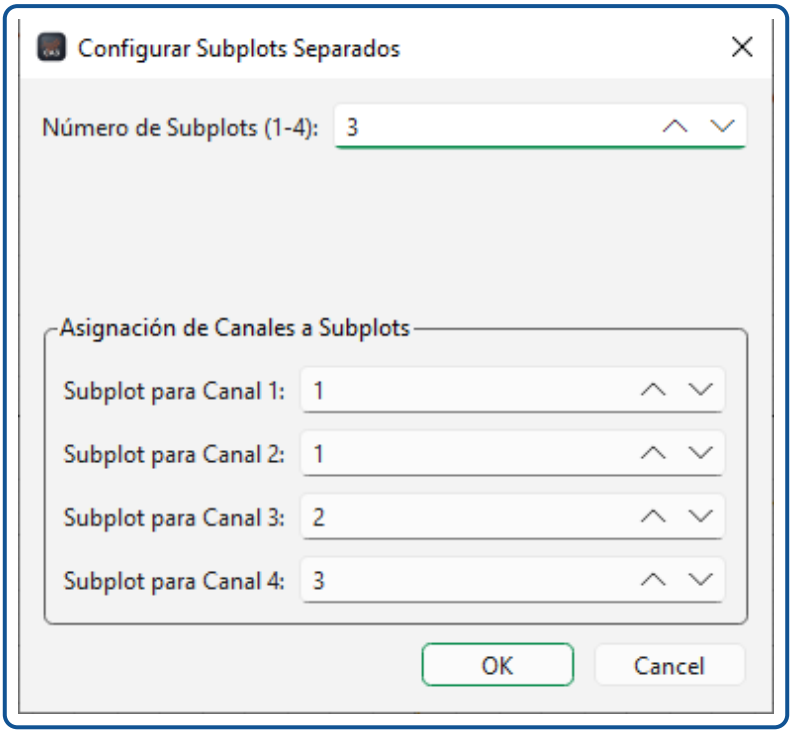


Figura 7.2: Diálogo modal de asignación flexible de canales a subplots.

7.2. Ventana Temporal y AutoScroll Horizontal

- **Ventana Temporal (X_{win}):** Configurable entre 0,5 s y 300,0 s. Determina la extensión horizontal visible del eje de tiempo.
- **AutoScroll X:** Cuando está activado, la vista de la gráfica sigue automáticamente la llegada de nuevas muestras manteniendo el límite derecho fijo en el último tiempo recibido t_{actual}. Si el usuario desplaza manualmente la gráfica con el mouse, el AutoScroll se desactiva automáticamente para permitir la inspección de datos pasados.

7.3. Escala Vertical (Autoscale vs Límites Manuales)

- **Autoscale Y (Activado):** Ajusta automáticamente los límites del eje vertical $[Y_{min}, Y_{max}]$ en cada cuadro para abarcar el rango dinámico de las señales visibles.
- **Autoscale Y (Desactivado):** Fijación manual de los ejes verticales en los valores numéricos especificados en las cajas **Ymin** y **Ymax** (por defecto $[-5{,}0\,\text{V}, +5{,}0\,\text{V}]$), evitando saltos ante picos aislados de ruido.

7.4. Personalización de Nombres de Canales

Haciendo clic en el botón **[Nombres]** del Panel 3, se despliega un diálogo modal que permite renombrar cada canal con etiquetas personalizadas (ej. "PWM", "Vout"). Estos nombres se actualizan instantáneamente en las leyendas de la gráfica, en los selectores de cursores y en las cabeceras de exportación.

7.5. Tasa de Refresco y Algoritmo de Decimación Gráfica

El motor gráfico basado en `pyqtgraph` actualiza la pantalla mediante un temporizador Qt configurado a 10 Hz (100 ms por cuadro).

Para garantizar un renderizado fluido sin saturar la CPU cuando la ventana visible contiene densidades elevadas de puntos, la clase `ScopePlotterWidget` aplica un algoritmo de decimación inteligente:

- **Umbral de Activación:** Se activa cuando la cantidad de muestras en la ventana excede las 2500 muestras (`max_display_points = 2500`).
- **Subsamplado con Preservación de Extremos:** Extrae muestras equiespaciadas con paso $S = N/2500$ e incluye explícitamente la última muestra recibida $(t[-1], y[-1])$, asegurando un desplazamiento continuo y preciso de la señal en tiempo real.

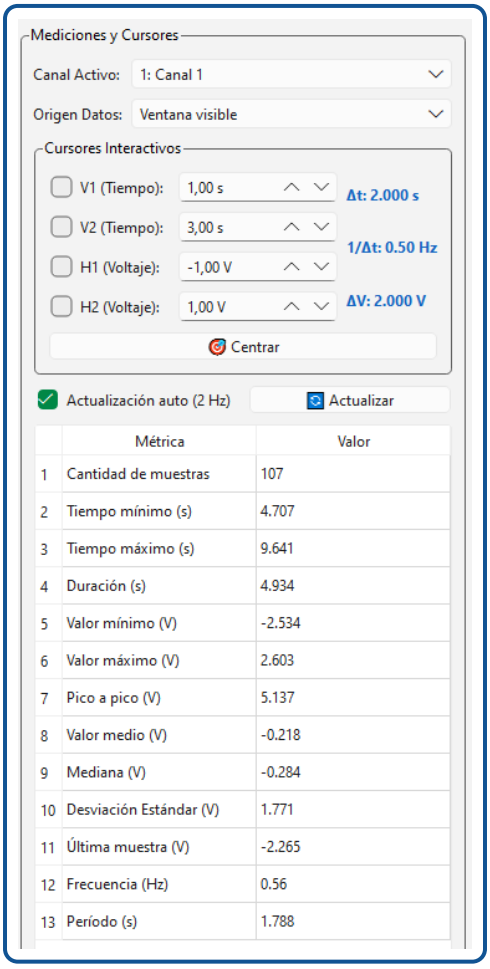
Advertencia Importante: La decimación afecta únicamente el dibujo en pantalla; los datos almacenados en el buffer circular y las exportaciones a archivos conservan el 100 % de las muestras originales sin pérdida alguna de información.

Capítulo 8

Cursores Interactivos y Análisis Estadístico

8.1. Sistema de Cursores Interactivos

El panel derecho **Mediciones y Cursores** provee cuatro cursores independientes que pueden superponerse sobre la gráfica:



	Métrica	Valor
1	Cantidad de muestras	107
2	Tiempo mínimo (s)	4.707
3	Tiempo máximo (s)	9.641
4	Duración (s)	4.934
5	Valor mínimo (V)	-2.534
6	Valor máximo (V)	2.603
7	Pico a pico (V)	5.137
8	Valor medio (V)	-0.218
9	Mediana (V)	-0.284
10	Desviación Estándar (V)	1.771
11	Última muestra (V)	-2.265
12	Frecuencia (Hz)	0.56
13	Período (s)	1.788

Figura 8.1: Panel de Mediciones y Cursores mostrando las métricas automáticas y lectura de cursores.

- **Cursores Verticales (V_1 y V_2):** Líneas verticales amarillas de posición temporal (t_1, t_2). Pueden desplazarse mediante cajas numéricas o arrastrándolos directamente con el botón izquierdo del mouse sobre la pantalla.
- **Puntos de Intersección sobre la Señal:** Al activar V_1 o V_2, se proyecta dinámicamente sobre la curva del canal considerado un **punto de contraste** (con el color asignado al cursor). El valor exacto de intersección (t, v) se calcula mediante **interpolación lineal en tiempo real**, permitiendo visualizar la amplitud instantánea de la señal.
- **Cursores Horizontales (H_1 y H_2):** Líneas horizontales cian de nivel de tensión (v_1, v_2).
- **Lecturas Diferenciales:**
  - **Intervalo de tiempo (Δt):** $\Delta t = |t_2 - t_1|$ [s].
  - **Frecuencia del intervalo ($1/\Delta t$):** $f = \frac{1}{\Delta t}$ [Hz].
  - **Diferencia de voltaje (ΔV):** $\Delta V = |v_2 - v_1|$ [V].
- **Botón Centrar (`[Centrar]`):** Ubica automáticamente los cursores V_1 y V_2 al 25 % y 75 % del ancho de la ventana visible actual.

8.2. Origen de Datos para Estadísticas

El selector **Origen Datos** determina qué conjunto de puntos es analizado por el motor estadístico (`MetricsEngine`):

1. **Ventana visible (por defecto):** Procesa únicamente los datos contenidos en el rango de tiempo visualizado en pantalla.
2. **Todos los datos:** Procesa la totalidad del buffer circular acumulado en memoria.
3. **Entre cursores V1-V2:** Restringe el cálculo estadístico estrictamente al intervalo de tiempo delimitado por los cursores verticales V_1 y V_2. Esta opción se **habilita automáticamente únicamente cuando ambos cursores V_1 y V_2 están activos**; si alguno se desactiva, el origen de datos vuelve automáticamente a *Ventana visible*.

8.3. Tabla de Métricas Automáticas

El cálculo numérico es llevado a cabo por la clase `MetricsEngine`. La tabla se actualiza automáticamente a una frecuencia de 2 Hz (cada 5 ticks del temporizador principal). La Tabla 8.1 detalla las 9 métricas principales presentadas en el panel.

Tabla 8.1: Métricas calculadas automáticamente por el motor estadístico (`MetricsEngine`).

Métrica en Tabla	**Símbolo / Unidad**	**Definición Matemática / Algoritmo**
Cantidad de muestras	N	Número de puntos válidos considerados.
Valor mínimo	V_{min} (V)	mín$(x[n])$.
Valor máximo	V_{max} (V)	máx$(x[n])$.
Pico a pico	V_{p2p} (V)	$V_{p2p} = V_{max} - V_{min}$.
Valor medio	$\bar{V}$ (V)	Componente continua (DC): $\bar{V} = \frac{1}{N}\sum x[n]$.
Mediana	V_{med} (V)	Valor percentil 50 de la muestra ordenada.
Desviación Estándar	σ (V)	Muestral $(N-1)$, equivale a la componente AC $(V_{rms,ac})$.
Frecuencia	f (Hz)	Estimación por cruce por cero con histéresis del 5 % de V_{p2p}.
Período	T (s)	$T = 1/f$.

8.4. Estimación de Frecuencia por Cruce por Cero con Histéresis

Para medir la frecuencia de oscilación de una señal ruidosa sin recurrir a transformadas de Fourier de alto costo computacional, `MetricsEngine.estimate_frequency()` implementa una máquina de estados con histéresis:

1. Se remueve la componente continua sustrayendo el valor medio: $x_{ac}[n] = x[n] - \bar{V}$.
2. Se establece un umbral de histéresis dinámico $V_{hys} = \text{máx}(10^{-4}, 0{,}05 \times V_{p2p})$.
3. Se detectan las transiciones del estado negativo al positivo únicamente cuando la señal supera $+V_{hys}$ tras haber descendido por debajo de $-V_{hys}$.
4. Se aplica **interpolación lineal** entre muestras consecutivas para determinar el tiempo exacto del cruce por cero t_{cross}.

5. El período T se obtiene como la **mediana de los períodos** entre cruces del mismo sentido consecutivos, haciéndolo extremadamente robusto frente a ruido impulsivo.

8.5. Significado Físico de la Desviación Estándar (σ) como Rizado AC

En ingeniería de control y procesamiento de señales:

- La **Desviación Estándar (σ)** calculada por `MetricsEngine` sustrae automáticamente la componente continua ($\bar{V}$) y evalúa la dispersión eficaz de la señal:

$$\sigma = \sqrt{\frac{1}{N-1}\sum_{n=1}^{N}(x[n]-\bar{V})^2}$$

- Físicamente, este valor coincide con la componente alterna eficaz o **rizado AC ($V_{rms,ac}$)**.
- En la evaluación de lazo cerrado, una σ elevada a la salida de un filtro o controlador revela la presencia de rizado remanente o oscilaciones de alta frecuencia (chattering). Para un análisis detallado sobre métricas y rizado en control clásico y robusto, consúltese [5] y [3].

Capítulo 9

Modo Trigger y Sincronización Avanzada

9.1. El Motor de Trigger (`TriggerEngine`)

El modo **Disparado (Trigger)** sustituye el desplazamiento continuo estilo registrador por un barrido sincronizado similar al de un osciloscopio de banco. Su función es congelar la señal en pantalla tomando como referencia un instante clave $t = 0$ definido por el cruce de un nivel de tensión.

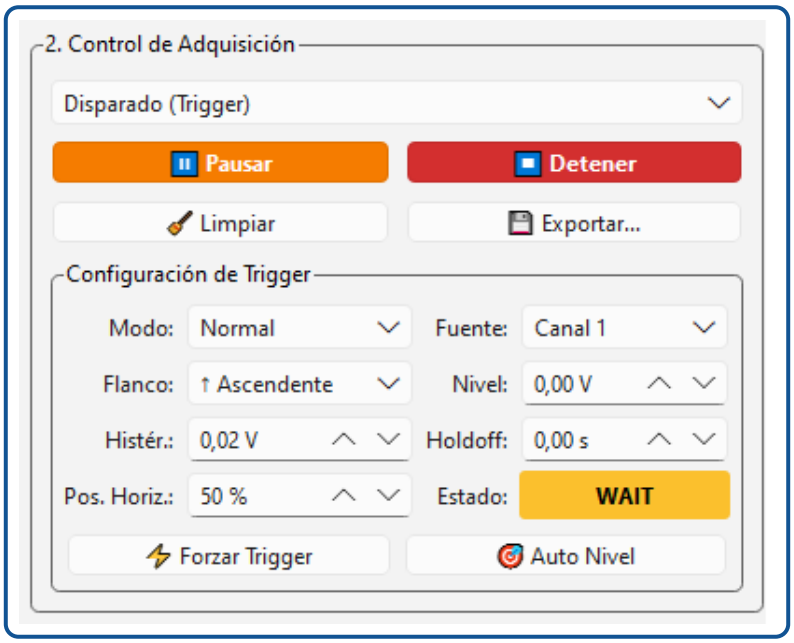


Figura 9.1: Controles del Panel de Trigger integrados en el Panel 2.

9.2. Parámetros de Configuración del Trigger

Tabla 9.1: Parámetros del Motor de Trigger.

Parámetro	Rango / Valores	Default	Efecto en el Disparo
Modo (`trig_mode`)	Normal, Single	Normal	Define si la pantalla re-dispara continuamente o frena en 1 captura.
Fuente (`source_channel`)	Ch 1 a Ch N	Ch 1	Canal analógico inspeccionado para detectar el cruce.
Flanco (`slope`)	Ascendente (↑), Descendente (↓)	Ascendente	Sentido de cruce por el nivel (positivo o negativo).
Nivel (`level`)	-100.0 V a 100.0 V	`0.0 V`	Valor de tensión en el cual se genera el disparo ($t = 0$).
Histéresis (`hysteresis`)	0.0 V a 10.0 V	`0.02 V`	Margen de ruido de armado para evitar disparos falsos.
Holdoff (`holdoff_s`)	0.0 s a 10.0 s	`0.0 s`	Tiempo mínimo de espera tras un disparo antes de rearmar.
Pos. Horiz. (`pretrigger_pct`)	0 % a 100 %	`50 %`	Porcentaje de la ventana temporal ubicado **antes** de $t = 0$.

9.3. Mecanismo de Anclaje de Ventana y Pre-trigger

La Figura 9.2 ilustra la distribución temporal durante una captura disparada:

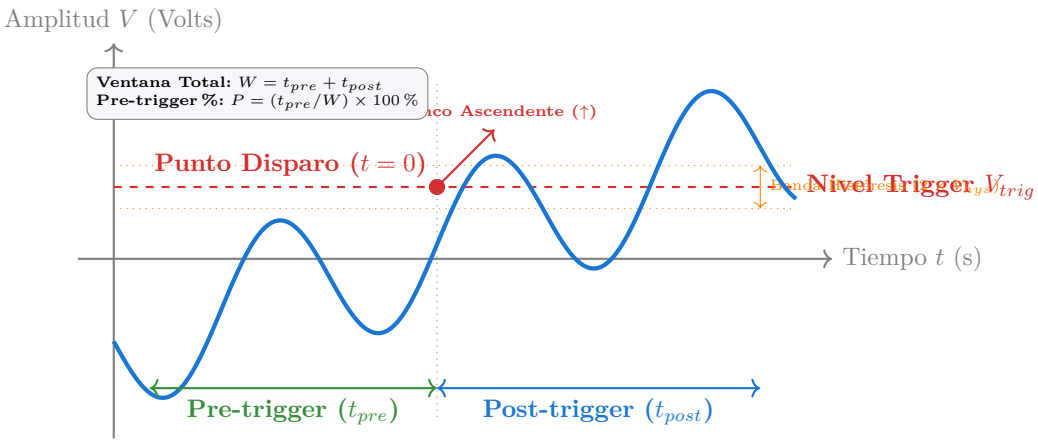


Figura 9.2: Mecanismo de anclaje de ventana temporal y cruce por nivel del Motor de Trigger.

- **Posición Horizontal al 50 % (Pre-trigger predeterminado):** El instante de disparo se ubica exactamente en el centro del eje X ($t = 0$). El 50 % izquierdo de la pantalla exhibe la historia previa al disparo (*pre-trigger*), mientras que el 50 % derecho muestra la evolución posterior (*post-trigger*).
- **Posición Horizontal al 20 %:** Útil para maximizar la visualización de la respuesta posterior a un escalón.

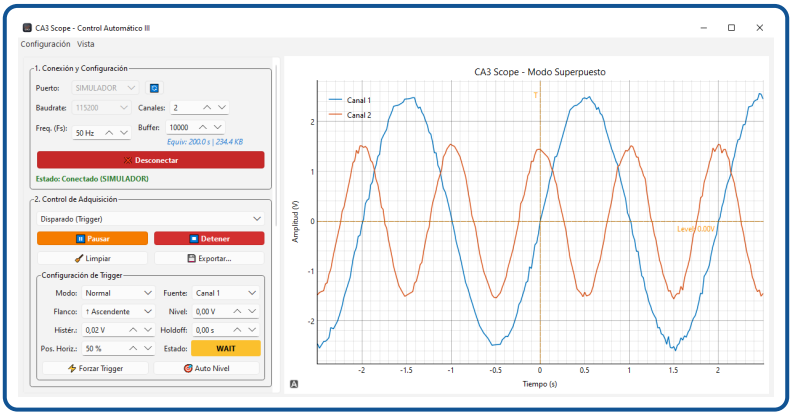


Figura 9.3: Pantalla del osciloscopio en estado Disparado (TRIG'D) mostrando los límites de pre-trigger y la línea horizontal de nivel.

9.4. Modos de Disparo: NORMAL vs SINGLE

- **Modo NORMAL:** El motor busca continuamente cruces de flanco. Cada vez que la señal satisface la condición de nivel y flanco, se actualiza el cuadro completo en pantalla. Si la señal deja de cruzar el nivel, la pantalla conserva la última captura retenida.
- **Modo SINGLE:** Al activar el modo de disparo único, el sistema arma el motor de trigger, aguarda el primer cruce válido, captura exactamente la ventana deseada ($t_{pre} + t_{post}$) y detiene automáticamente la adquisición pasando al estado **STOP**. Es el modo fundamental para registrar eventos transitorios únicos.

9.5. Indicadores de Estado del Trigger (Badges)

El estado del motor de trigger se visualiza mediante un badge en el panel:

- **WAIT**: El motor está armado y aguarda a que la señal cruce el nivel configurado.
- **TRIG'D**: Se detectó un cruce válido y se completó la extracción del cuadro.
- **FORCE**: Se ejecutó un disparo forzado manual mediante el botón [**Forzar Trigger**].
- **HOLD**: En período de espera holdoff tras un disparo.
- **STOP**: El motor de trigger está desarmado o la adquisición se encuentra detenida.

9.6. Botones de Acción de Trigger

- **Botón Auto Nivel (`[Auto Nivel]`):** Calcula el valor medio ($\bar{V}$) del canal seleccionado como fuente dentro del buffer circular y lo asigna inmediatamente como el nuevo **Nivel de Trigger**.
- **Botón Forzar Trigger (`[Forzar Trigger]`):** Fuerza la captura inmediata del cuadro con la estampa de tiempo actual sin esperar el cruce por nivel.

Capítulo 10

Exportación de Datos y Capturas

10.1. El Diálogo de Exportación

Al hacer clic en el botón [**Exportar...**] en el Panel 2, se abre el diálogo modal `ExportDialog`, el cual permite seleccionar el formato de destino y el intervalo temporal a recortar.

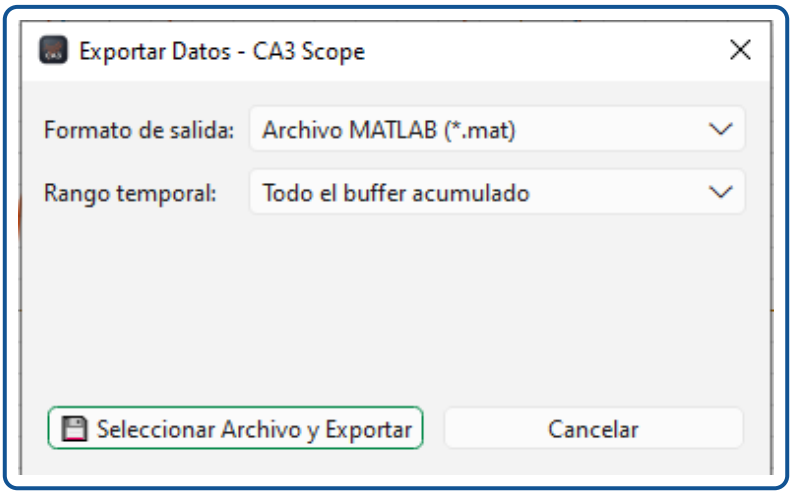


Figura 10.1: Diálogo modal de exportación seleccionando formato de salida y rango temporal.

10.2. Formatos de Archivo Soportados

Archivo CSV (*.csv): Exporta una matriz plana de texto legible delimitada por comas con los encabezados `time, Canal_1, Canal_2, ...`.

```
time,Canal_1,Canal_2
0.000000,2.500000,1.230000
0.020000,2.520000,1.210000
0.040000,2.550000,1.180000
```

Archivo MATLAB (*.mat): Exporta un archivo binario nativo en formato MATLAB v5/v7 compatible con scipy.io.savemat. Contiene un diccionario de variables estructurado:

- `t`: Vector columna de tiempos de dimensión $N \times 1$ (`double`).
- `data`: Matriz de muestras de dimensión $N \times N_{canales}$ (`double`).

- `channel_names`: Cell array con las etiquetas de los canales.
- `sample_time`: Período de muestreo configurado $T_s = 1/F_s$ (`double`).
- `export_date`: Estampa de fecha y hora de la exportación.
- `source_port`: Nombre del puerto serie o simulador emisor.

Captura Imagen PNG (*.png): Guarda una imagen PNG del widget gráfico activo (`ScopePlotterWidget`) en alta resolución.

10.3. Opciones de Recorte de Rango Temporal

Para evitar exportar archivos masivos innecesarios, la clase `DataExporter.crop_data()` permite recortar la matriz de datos según las siguientes modalidades:

1. **Todo el buffer acumulado (`'all'`):** Guarda la totalidad de las muestras almacenadas en el `RingBuffer`.
2. **Ventana visible actual (`'visible'`):** Guarda únicamente las muestras contenidas entre t_{min} y t_{max} de la vista actual en pantalla.
3. **Rango entre cursores V1-V2 (`'cursors'`):** Recorta estrictamente las muestras comprendidas entre la posición de V_1 y V_2. Esta opción se habilita en el diálogo únicamente si ambos cursores verticales están activos.

10.4. Script de Importación y Análisis en MATLAB

A continuación se proporciona un script de referencia en MATLAB para cargar automáticamente el archivo `.mat` generado por CA3Scope-Py y preparar la estructura de datos requerida por la System Identification Toolbox [8]:

```
% Script de Carga de Datos CA3Scope-Py en MATLAB
clear; clc; close all;

% 1. Cargar archivo .mat exportado
filename = 'ca3_scope_datos.mat';
load(filename);

% 2. Extraer vectores
time_vector = t;          % Vector de tiempo (N x 1)
u = data(:, 1);           % Entrada (Canal 1)
y = data(:, 2);           % Salida (Canal 2)

% 3. Graficar senales importadas
figure('Name', 'Datos Importados de CA3Scope-Py');
plot(time_vector, u, 'b--', 'LineWidth', 1.5); hold on;
plot(time_vector, y, 'r-', 'LineWidth', 2.0);
grid on;
xlim([time_vector(1); time_vector(end)]);
xlabel('Tiempo (segundos)');
ylabel('Amplitud (Volts)');
legend(channel_names{1}, channel_names{2});
title(['Ensayo Exportado el: ' export_date]);

% 4. Crear objeto iddata para System Identification Toolbox
Ts = sample_time;
datos_ident = iddata(y, u, Ts);
```

```
datos_ident.InputName = channel_names{1};
datos_ident.OutputName = channel_names{2};

disp('Objeto iddata creado exitosamente.');
```

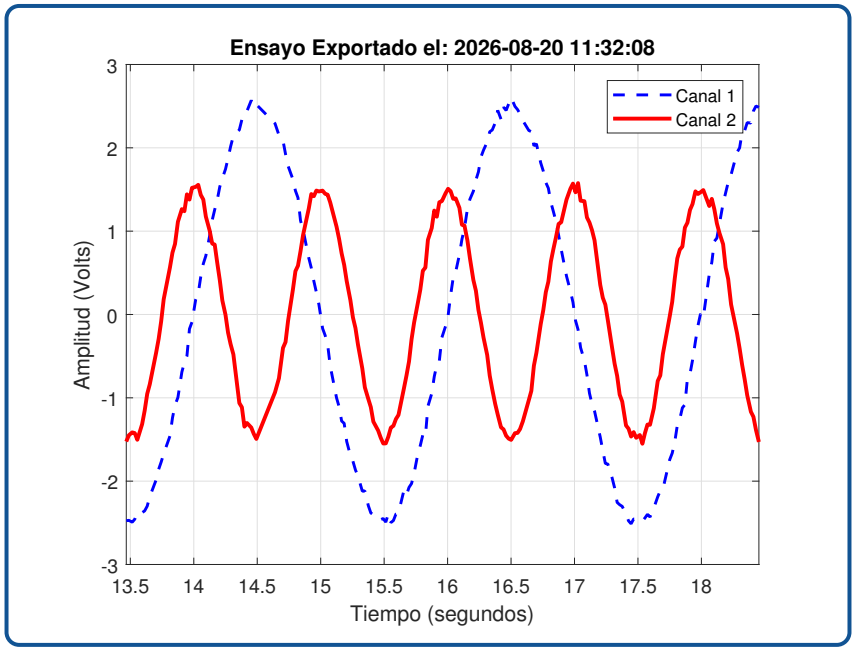


Figura 10.2: Gráfica generada en MATLAB al importar y plotear el archivo .mat con el script de carga.

Capítulo 11

Técnicas de Mediciones y Metodología de Trabajo

Este capítulo presenta las metodologías de medición y las técnicas instrumentales fundamentales utilizando CA3Scope-Py. Para facilitar el entrenamiento sin requerir hardware externo, todas las fichas técnicas se ejecutan directamente sobre el **Simulador Interno de Señales**.

11.1. Fichas de Técnica Instrumental

Ficha 1: Captura de Transitorios Únicos (Modo Single + Pre-trigger)

Objetivo: Registrar eventos que ocurren una sola vez (como respuestas al escalón o maniobras de conmutación) sin perder la información previa al disparo.

- **Configuración:** En el Panel 1 seleccioná el puerto `SIMULADOR` y en el Panel 2 cambiá el modo a **Disparado (Trigger)**.
- **Ajuste del Trigger:** Seleccioná modo **Single**, flanco ascendente y un porcentaje de pre-trigger del 20 % al 50 %.
- **Ejecución:** Al iniciar la captura, el sistema mostrará el estado **WAIT**. Al producirse el cruce por el nivel ajustado, congelará automáticamente la pantalla en **STOP** registrando la historia previa ($t < 0$) y la evolución posterior ($t > 0$).

Ficha 2: Medición de Δt y ΔV con Cursores Interactivos

Objetivo: Medir con precisión diferencias temporales y de amplitud sobre cualquier canal visible.

- **Procedimiento:** En el panel derecho de **Mediciones y Cursores**, activá las casillas de los cursores verticales V_1, V_2 o horizontales H_1, H_2.
- **Ajuste:** Desplazá los cursores arrastrándolos con el mouse o ingresando valores numéricos exactos en sus campos.
- **Lectura:** Las etiquetas superiores actualizarán de forma instantánea las diferencias $\Delta t = |t_2 - t_1|$, su inversa $1/\Delta t$ y el salto de tensión $\Delta V = |v_2 - v_1|$. Hacé clic en **[Centrar]** si deseás reubicar los cursores en la ventana visible.

Ficha 3: Medición de Frecuencia y Período

Objetivo: Determinar la frecuencia fundamental de oscilaciones periódicas ruidosas o limpias.

- **Método Automático:** Inspeccioná la tabla de estadísticas automáticas en las filas **Frecuencia (Hz)** y **Período (s)**. El algoritmo de cruce por cero con histéresis e interpolación calculará el valor medio robusto.

- **Método Manual:** Posicioná el cursor V_1 en un pico o cruce de flanco y el cursor V_2 en el siguiente pico o cruce del mismo sentido. El valor de $1/\Delta t$ indicará la frecuencia calculada por cursores.

Ficha 4: Cuantificación de Rizado y Componente Alterna ($V_{rms,ac}$)

Objetivo: Evaluar el nivel de ruido, rizado o chattering presente en una señal sobre una componente continua.

- **Procedimiento:** Seleccioná el canal a evaluar como **Canal Activo** en el panel derecho.
- **Interpretación:** Consultá la fila **Desviación Estándar (V)** de la tabla de métricas. Matemáticamente, la desviación estándar muestral (σ) representa la energía de la componente alterna o rizado puro ($V_{rms,ac}$), descartando el valor medio continuo (componente DC).

Ficha 5: Observación de Saturación y Recorte de Señales

Objetivo: Identificar la presencia de límites físicos o restricciones de hardware en señales de control o variables de proceso.

- **Procedimiento:** Graficá la señal en modo superpuesto o subplots y ajustá los límites verticales Y manualmente o mediante Autoscale.
- **Diagnóstico:** Una meseta horizontal plana sostenida en el valor máximo (ej. 5,0 V) o mínimo (ej. 0,0 V) indica que la variable ha alcanzado un límite físico de saturación (restricción de actuador o recorte de amplificador).

Ficha 6: Exportación de Rango Temporal y Análisis Posterior

Objetivo: Extraer datos experimentales limpios para procesamiento en MATLAB® o GNU Octave.

- **Procedimiento:** Presioná el botón [**Exportar...**] en el Panel 2.
- **Selección de Rango:** Elegí el rango temporal adecuado (**Ventana visible** o **Entre cursores V1-V2**) para evitar exportar tramos no deseados de régimen permanente.
- **Formato:** Exportá a formato **MATLAB (*.mat)** y utilizá el script de carga provisto en el Capítulo 10.

11.2. Mapeo: Necesidad de Medición vs Herramienta del Instrumento

La Tabla 11.1 resume el recurso más directo de CA3Scope-Py ante cada necesidad de análisis en las prácticas de laboratorio.

Tabla 11.1: Matriz de relación entre necesidades de medición y herramientas de CA3Scope-Py.

Necesidad de Medición	**Herramienta de CA3Scope-Py**	**Modo / Panel Sugerido**
Respuesta a escalón / Evento único	Modo Disparado (Single) + Pre-trigger	Panel 2 / Panel Trigger (20 % pre-trig).
Sobrepaso máximo (M_p)	Cursores Horizontales $(H_1, H_2) \rightarrow \Delta V$	Panel Mediciones / Cursores.
Tiempo de asentamiento (t_s)	Cursores Verticales $(V_1, V_2) \rightarrow \Delta t$	Panel Mediciones / Cursores.
Período (T) y Frecuencia (f)	Tabla de Métricas o Cursores V_1, V_2	Panel Mediciones (Frec. / Período).
Rizado / Componente Alterna	Desviación Estándar $(\sigma = V_{rms,ac})$	Panel Mediciones / Tabla de Métricas.
Saturación de actuadores	Modo Subplots + Límites Y	Panel 3 (Visualización) / Subplots.
Identificación de sistemas	Exportación de rango temporal a MAT	Panel 2 / Diálogo Exportar → MATLAB.

11.3. Recomendaciones según Temática de la Asignatura

Identificación de Sistemas: Resulta de especial utilidad la captura en modo Single Shot con pre-trigger para registrar el estado de régimen permanente inicial previo al escalón, exportando únicamente la ventana temporal de interés a formato MAT.

Control PID Clásico: La visualización en Subplots separados permite aislar la acción de control de la salida del proceso, mientras que los cursores horizontales y verticales facilitan la determinación rápida del sobrepaso y tiempos característicos.

Control Predictivo (MPC): El modo Subplots combinados permite observar claramente la coincidencia de las restricciones de saturación activas en la acción de control con las dinámicas de respuesta de la salida.

Sistemas No Lineales: Para la caracterización de ciclos límite auto-sostenidos en sistemas con relé o histéresis, la combinación del cálculo automático de frecuencia con la medición directa de la amplitud pico a pico provee la información necesaria para el análisis por función descriptiva [7].

Capítulo 12

Preguntas Frecuentes y Resolución de Problemas

12.1. Problemas Frecuentes y Soluciones (Troubleshooting)

Problemas de conexión del puerto (puerto no detectado, permisos, tramas corruptas):

Ver la Sección 5.6, pág. 17 para resolver problemas de puerto ocupado o bloqueado (*PermissionError*), falta de controladores del puerto serie virtual, desajustes de baudrate o tramas corruptas.

4. La escala de tiempo de la gráfica se desplaza más rápido o más lento que el tiempo real:

Causa: Desajuste entre la frecuencia de muestreo configurada en la GUI (**Freq. Fs**) y el intervalo de muestreo real del firmware (**Ts**).
Solución:

- La base de tiempo en segundos es generada en la PC. Si el firmware envía datos cada 20 ms (50 Hz), ajustá el campo **Freq. (Fs)** en la aplicación a exactamente 50 Hz.
- Comprobá que el código del microcontrolador utilice un bucle no bloqueante basado en `micros()` (evitando la función `delay()`) para prevenir derivas temporales.

5. Mensaje de advertencia de Windows SmartScreen al abrir el programa:
Causa: La aplicación ejecutable distribuida no posee una firma digital comercial paga.
Solución: En la ventana azul de advertencia de Windows, hacé clic en el enlace **"Más información"** y luego seleccioná el botón **"Ejecutar de todos modos"**.

12.2. Resumen de Preguntas Frecuentes (FAQ)

¿Cómo ejecutar el programa en una PC sin Python ni permisos de administrador?

Descargá el archivo comprimido oficial (`CA3Scope.zip`) desde el Moodle de la cátedra, descomprimí la carpeta completa en cualquier ubicación (disco o pendrive USB) y ejecutá directamente `CA3Scope.exe`. No requiere instalación previa ni permisos de administrador.

¿Puedo usar CA3Scope-Py con placas ESP32, STM32 o Raspberry Pi Pico?
Sí, la aplicación es agnóstica al hardware emisor. Cualquier dispositivo que transmita tramas de texto separadas por comas vía UART USB es compatible.

¿Cuál es la frecuencia de muestreo máxima recomendada?
Para enlaces serie estándar a 115200 baudios con 2 canales, la frecuencia óptima se sitúa

entre 50 Hz y 200 Hz. Para frecuencias superiores, se recomienda utilizar baudrates de 230400 baudios.

¿Dónde se guardan las configuraciones de la aplicación?
Las preferencias se almacenan o cargan mediante archivos JSON desde el menú **Configuración → Guardar Configuración... / Cargar Configuración...**.

Apéndice A

Guía para Desarrolladores (Instalación desde Código Fuente)

Este anexo está destinado exclusivamente a docentes, desarrolladores o alumnos que deseen modificar el código fuente de CA3Scope-Py, ejecutar la aplicación en entornos de desarrollo o empaquetar una nueva versión ejecutable.

A.1. Requisitos de Desarrollo

- **Python:** Versión 3.10 o superior (64-bit).
- **Librerías principales:** PySide6, NumPy, SciPy, PySerial y PyQtGraph [2].
- **Entorno de Desarrollo:** VS Code, PyCharm o terminal de comandos.
- **Gestor de paquetes:** `pip` actualizado.

A.2. Configuración del Entorno de Desarrollo

Para preparar el entorno e instalar las dependencias:

1. Descargar o clonar el repositorio e ingresar a la carpeta raíz del proyecto.
2. Crear y activar un entorno virtual (`venv`):

   En Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1

```

   En Linux / macOS (Bash):

```
python3 -m venv .venv
source .venv/bin/activate

```

3. Instalar las librerías requeridas especificadas en `requirements.txt`:

```
pip install --upgrade pip
pip install -r requirements.txt

```

4. Ejecutar la aplicación en modo desarrollo:

```
python app/main.py

```

A.3. Construcción del Ejecutable Portable con PyInstaller

Para generar la distribución ejecutable de Moodle (`CA3Scope.exe` junto a su carpeta `_internal/`), se utiliza el script provisto `build_exe.py`:

```
python build_exe.py
```

Este script invoca PyInstaller configurando las opciones necesarias (`-onedir`, recolección de DLLs de PySide6 y shiboken6, inclusión del ícono y manifiesto) y deposita el resultado en el directorio `dist/CA3Scope/`.

A.4. Notas de implementación del Buffer y Gráficas

La estructura interna de almacenamiento y el renderizado gráfico de CA3Scope-Py han sido diseñados bajo los siguientes criterios de bajo nivel para maximizar la velocidad y eficiencia en el procesamiento de datos:

- **Pre-asignación de Matrices en NumPy:** El `RingBuffer` reserva al conectarse una matriz continua en memoria RAM utilizando flotantes de precisión doble de 64 bits (`np.float64`), con dimensiones de $(N_{max}, N_{canales} + 1)$. La primera columna se reserva para la estampa temporal monotónica.
- **Complejidad Temporal de Escritura $O(1)$:** La inserción de nuevas muestras se efectúa mediante aritmética modular del índice del cursor de escritura, lo cual evita el copiado en memoria de elementos y garantiza que el tiempo de agregado sea constante e independiente del tamaño del buffer.
- **Extracción por Filtro de Máscara Booleana:** La consulta de un intervalo temporal para gráficos o exportación se realiza aplicando una operación de indexación booleana vectorial en NumPy:

```
mask = (timestamps >= t_min) & (timestamps <= t_max)
return timestamps[mask], data[mask]

```

  Esta técnica vectorial nativa es significativamente más veloz que la búsqueda binaria mediante algoritmos secuenciales de búsqueda como `np.searchsorted`.
- **Módulo de Parseo de Protocolos (`protocol.py`):** Las funciones `parse_line()` y `parse_binary_frame()` se encargan de la desfragmentación y validación de límites. En modo binario, `struct.unpack(«nh", raw_payload)` desempaqueta eficientemente los enteros de 16 bits little-endian a tipos nativos en C antes de insertarlos en el buffer circular NumPy.
- **Algoritmo de Decimación de Pantalla:** Para mantener la tasa de refresco a 10 Hz constantes sin saturar la CPU cuando la ventana horizontal visualizada abarca una cantidad de muestras superior a 2500 (`max_display_points = 2500`), el sistema aplica un filtrado por paso de rebanado (*slicing*) de NumPy. El vector a graficar se subsamplea dinámicamente mediante `t[::step]` asegurando que el último elemento recibido (`t[-1]` y `y[-1]`) se agregue manualmente al final para evitar discontinuidades gráficas.

Bibliografía

[1] AVR Libc Development Team. *AVR Libc Reference Manual*, 2025. https://www.nongnu.org/avr-libc/user-manual/.

[2] Luke Campagnola and contributors. *PyQtGraph: Scientific Graphics and GUI Library for Python*, 2025. Disponible en línea: https://www.pyqtgraph.org/.

[3] John C. Doyle, Bruce A. Francis, and Allen R. Tannenbaum. *Feedback Control Theory*. Macmillan Publishing Company, 1992.

[4] Microchip Technology Inc. *ATmega328P Data Sheet*, 2023. DS40002061, https://www.microchip.com/en-us/product/ATMEGA328P.

[5] Katsuhiko Ogata. *Modern Control Engineering*. Prentice Hall, 5th edition, 2010.

[6] PlatformIO Team. *PlatformIO Core Documentation*, 2026. Disponible en línea: https://docs.platformio.org/.

[7] Jean-Jacques E. Slotine and Weiping Li. *Applied Nonlinear Control*. Prentice Hall, 1991.

[8] Arun K. Tangirala. *Principles of System Identification: Theory and Practice*. CRC Press, 2014.